

Metamagic Feats Implementation Plan — D&D 3.5e PHB

Status: Design Complete — Ready for Implementation

Target: `/home/ubuntu/dnd35prototype`

Prerequisite for: Metamagic Rods, Rod System, Advanced Spellcasting UI

D&D 3.5e Source: Player's Handbook Chapter 5, pp. 87–92

Table of Contents

1. [Overview](#)
 2. [System Architecture](#)
 3. [Existing Codebase Analysis](#)
 4. [Core Mechanics](#)
 5. [All 9 PHB Metamagic Feats](#)
 6. [Technical Implementation](#)
 7. [Rod Integration Design](#)
 8. [Stacking Rules](#)
 9. [Spontaneous vs Prepared Casting](#)
 10. [Implementation Phases](#)
 11. [Testing Requirements](#)
 12. [Appendix: Quick Reference Tables](#)
-

1. Overview

What Are Metamagic Feats?

Metamagic feats allow spellcasters to modify their spells at the time of casting, trading higher-level spell slots for enhanced effects. They represent a caster's mastery over the fundamental nature of magic — stretching, compressing, amplifying, or reshaping spells beyond their normal parameters.

Why They're Needed

1. **Rod System Dependency:** Metamagic Rods (Lesser/Normal/Greater) apply metamagic effects to spells without the slot-level increase. The metamagic system must exist before rods can reference it.
2. **Spellcaster Depth:** Metamagic is a core strategic layer for spellcasters — choosing when to empower vs. quicken vs. extend a spell is a defining decision.
3. **Wizard Bonus Feats:** Wizards can select metamagic feats as their bonus feats at levels 5, 10, 15, 20 — these are already defined in `FeatDefinitions.cs` and marked `IsWizardBonus = true`.
4. **AI Spell Strategy:** The AI spellcasting strategist (`AISpellcastingStrategist`) needs metamagic awareness to make intelligent casting decisions.

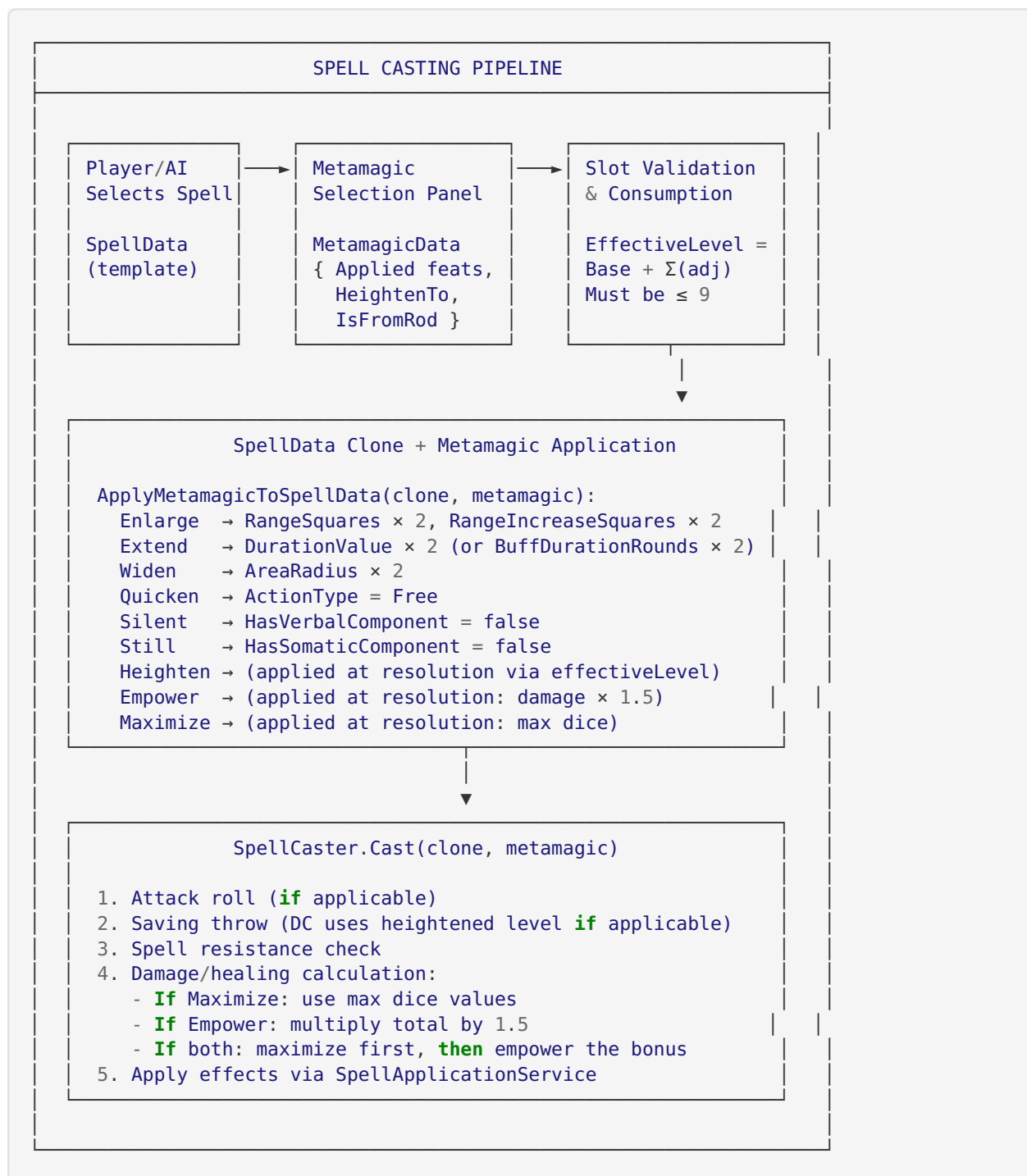
5. **Combat UI:** The spell selection UI needs a metamagic toggle panel to let players apply metamagic before casting.

Design Principles

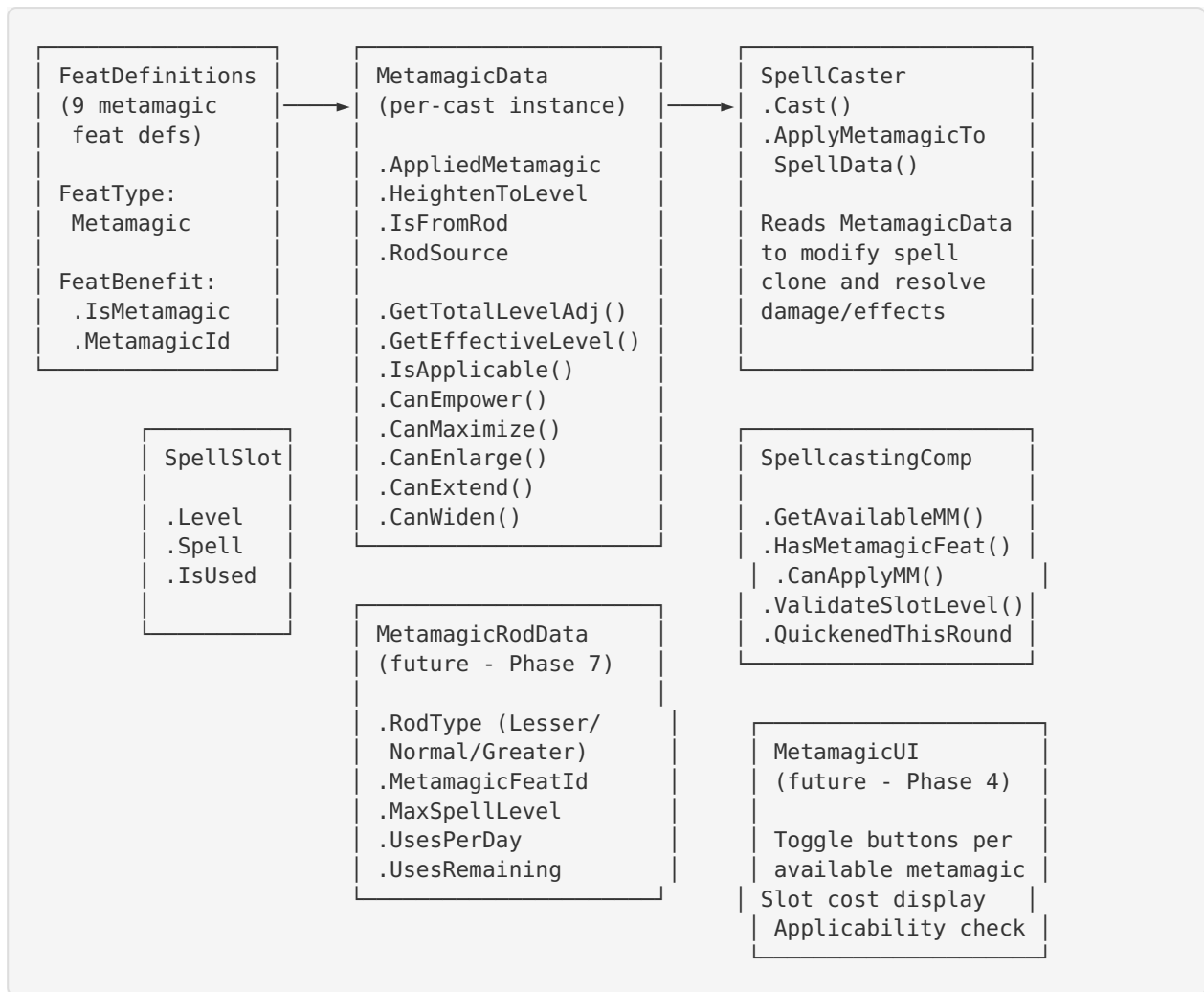
- **PHB Accuracy:** All mechanics match D&D 3.5e Player's Handbook exactly
 - **Non-Destructive:** Metamagic modifies cloned SpellData, never the original template
 - **Rod-Ready:** Architecture supports "apply metamagic without slot increase" from day one
 - **UI-Agnostic:** Core logic is pure data/math; UI is a separate concern
 - **Stackable:** Multiple metamagic feats can be applied to a single spell
-

2. System Architecture

High-Level Data Flow



Component Relationships



3. Existing Codebase Analysis

What Already Exists

The codebase has **substantial** metamagic infrastructure already built. This plan enhances and completes it.

MetamagicData.cs (262 lines) — FULLY IMPLEMENTED

- `MetamagicFeatId` enum with all 9 feats
- `MetamagicData` class with `AppliedMetamagic` HashSet
- `GetTotalLevelAdjustment()` — calculates cumulative slot increase
- `GetEffectiveSpellLevel()` — base + adjustments
- `GetLevelAdjustment()` — per-feat level cost (Empower +2, Enlarge +1, etc.)
- `IsApplicable()` — static checks per feat (Empower needs damage/healing, Enlarge needs range > 0, etc.)
- `CanEmpower()`, `CanMaximize()`, `CanEnlarge()`, `CanExtend()`, `CanWiden()` — individual applicability checks
- `GetDisplayName()`, `GetShortEffect()`, `GetSummary()` — display helpers

- `GetIdFromFeatName()` / `GetFeatName()` — string↔enum conversion
- `AllMetamagicFeats` static array

SpellCaster.cs — METAMAGIC RESOLUTION IMPLEMENTED

- `ApplyMetamagicToSpellData(SpellData, MetamagicData)` — pre-cast modifications:
- Enlarge: doubles `RangeSquares` and `RangeIncreaseSquares`
- Extend: doubles `DurationValue` (or legacy `BuffDurationRounds`)
- Widen: doubles `AreaRadius`
- Quicken: sets `ActionType = SpellActionType.Swift`
- Silent: sets `HasVerbalComponent = false`
- Still: sets `HasSomaticComponent = false`
- `Cast()` resolution checks:
- `isEmpowered` → multiplies damage/healing by 1.5
- `isMaximized` → uses max dice values
- `isHeightened` → increases effective spell level for DC calculation
- `SpellResult.Metamagic` — stores `MetamagicData` on result for combat log

FeatDefinitions.cs — ALL 9 FEATS DEFINED

- `DefineMetamagicFeats()` registers all 9 metamagic feats
- Each feat has: name, PHB description, `FeatType.Metamagic`, `IsWizardBonus = true`
- Each feat's `FeatBenefit` has: `IsMetamagic = true`, `MetamagicId` set
- All registered in static feat dictionary

Feat.cs — DATA STRUCTURES READY

- `FeatType.Metamagic` enum value exists
- `FeatBenefit.IsMetamagic` and `FeatBenefit.MetamagicId` fields exist
- `FeatDefinition` class supports prerequisites, wizard bonus flag

SpellSlot.cs — SLOT SYSTEM READY

- Individual `SpellSlot` objects with `Level`, `PreparedSpell`, `IsUsed`
- Domain slot and specialist slot tracking
- Multi-class caster support via `CasterClassName`

SpellcastingComponent.cs — PARTIAL SUPPORT

- `HasQuickenedThisRound` field exists (quicken limit enforcement)
- `KnownSpells` and `SpellSlots` lists exist
- Spell preparation and consumption logic exists
- No metamagic-aware slot consumption yet

What’s Missing ❌

Component	Status	What’s Needed
Metamagic-aware slot consumption	❌ Not built	Consume higher-level slot when metamagic applied
Character metamagic feat query	❌ Not built	<code>GetAvailable-MetamagicFeats(CharacterStats)</code>
Prepared caster metamagic prep	❌ Not built	Wizard prepares metamagic version in higher slot
Spontaneous caster casting time	❌ Not built	Sorcerer metamagic = full-round action (not standard)
Metamagic selection UI	❌ Not built	Toggle panel during spell selection
Rod integration hooks	❌ Not built	<code>IsFromRod</code> flag to bypass slot increase
AI metamagic strategy	❌ Not built	When AI should apply which metamagic
Empower + Maximize stacking	⚠️ Partial	Both checked independently; stacking order unclear
Heighten DC enforcement	⚠️ Partial	Level adjustment works, DC calc uses it, but no UI
Cantrip metamagic	❌ Not handled	Level 0 + metamagic = still uses slot (level 0 + adj)
Level 9 cap validation	❌ Not enforced	Effective level must not exceed 9

4. Core Mechanics

4.1 Spell Slot Level Increase

When a caster applies metamagic to a spell, it consumes a higher-level spell slot. The spell’s **actual** level doesn’t change (for dispel checks, counterspelling, etc.), but the **slot** consumed is higher.

Effective Slot Level = Base Spell Level + Σ (Metamagic Level Adjustments)

Example: Fireball (3rd) + Empower (+2) + Maximize (+3) = 8th-level slot

Constraint: Effective slot level **cannot exceed 9**. If the combined metamagic would push it above 9, the combination is invalid.

Exception: Level 0 spells (cantrips) can have metamagic applied. A cantrip with +1 metamagic uses a 1st-level slot (not a 0th-level slot).

4.2 Slot Level Increase Table

Metamagic Feat	Level Adjustment	Slot for Lv1 Spell	Slot for Lv3 Spell	Slot for Lv5 Spell
Enlarge Spell	+1	2nd	4th	6th
Extend Spell	+1	2nd	4th	6th
Silent Spell	+1	2nd	4th	6th
Still Spell	+1	2nd	4th	6th
Empower Spell	+2	3rd	5th	7th
Maximize Spell	+3	4th	6th	8th
Widen Spell	+3	4th	6th	8th
Quicken Spell	+4	5th	7th	9th
Heighten Spell	variable	target	target	target

4.3 Casting Time Changes

Prepared casters (Wizard, Cleric): Metamagic **does not** change casting time. The metamagic is applied during preparation. The actual casting is normal.

Spontaneous casters (Sorcerer, Bard): Applying metamagic **increases casting time**:

- Standard action spell → **Full-round action**
- Full-round action spell → Casting time becomes **1 full round + 1 standard action** (effectively 2 rounds)
- **Exception:** Quicken Spell always makes it a free action regardless of caster type

CASTING TIME RULES BY CASTER TYPE		
Original Action	Prepared Caster	Spontaneous Caster
Standard Full-Round + Quicken Spell	Standard Full-Round Free Action	Full-Round 1 Full Round (next turn) Free Action

4.4 Preparation vs Selection

Prepared casters: Metamagic is chosen **during spell preparation** (morning ritual). A Wizard must decide at preparation time to put “Empowered Fireball” in a 5th-level slot. They cannot decide at cast time.

Spontaneous casters: Metamagic is chosen **at cast time**. A Sorcerer decides when casting whether to empower their Fireball, but it takes a full-round action and uses a 5th-level slot.

4.5 Component Modifications

- **Silent Spell:** Removes the verbal component. Spell can be cast in a silence effect.
- **Still Spell:** Removes the somatic component. Spell can be cast while grappled or in armor without arcane spell failure.
- Combining Silent + Still removes both → spell can be cast with no observable external signs (only mental).

5. All 9 PHB Metamagic Feats

5.1 Empower Spell

Property	Value
PHB Page	p.87
Level Adjustment	+2
Prerequisite	None
Applicable To	Spells with variable numeric effects (damage, healing, bonus amounts)
Effect	All variable numeric effects increased by 50% (round down)
Stacks With	Maximize Spell (maximize first, then add 50% of max as bonus)

Detailed Mechanics:

- “Variable numeric effects” = anything determined by dice rolls
- Fireball (5d6) → empowered = 5d6 + 50% of roll (if rolled 18, becomes 18 + 9 = 27)
- Does NOT affect: range, duration, targets, saving throw DC
- Does NOT affect non-variable bonuses (e.g., +2 bonus from Bull’s Strength stays +2)
- Saving throw bonuses from spells like resistance (+1) are NOT variable

Implementation (existing in `SpellCaster.Cast()`):


```

if (isEmpowered) {
    totalDamage = Mathf.FloorToInt(totalDamage * 1.5f);
    totalHealing = Mathf.FloorToInt(totalHealing * 1.5f);
}

```

Applicability Check (existing in `MetamagicData`):

```

public static bool CanEmpower(SpellData spell) {
    return spell.EffectType == SpellEffectType.Damage || spell.EffectType == SpellEffectType.Healing;
}

```

5.2 Enlarge Spell

Property	Value
PHB Page	p.88
Level Adjustment	+1
Prerequisite	None
Applicable To	Spells with range of close, medium, or long (not touch, personal, or self)
Effect	Double the spell's range

Detailed Mechanics:

- Close range (25 ft + 5 ft/2 levels) → doubled
- Medium range (100 ft + 10 ft/level) → doubled
- Long range (400 ft + 40 ft/level) → doubled
- Does NOT affect touch spells or personal-range spells
- Does NOT affect area of effect (that's Widen Spell)

Implementation (existing in `SpellCaster.ApplyMetamagicToSpellData()`):

```

if (metamagic.Has(MetamagicFeatId.EnlargeSpell) && spell.RangeSquares > 0) {
    spell.RangeSquares *= 2;
    spell.RangeIncreaseSquares *= 2; // Also double the per-level scaling
}

```

5.3 Extend Spell

Property	Value
PHB Page	p.88
Level Adjustment	+1
Prerequisite	None
Applicable To	Spells with duration > instantaneous (not concentration, instantaneous, or permanent)
Effect	Double the spell's duration

Detailed Mechanics:

- 1 round/level → 2 rounds/level
- 1 min/level → 2 min/level
- 10 min/level → 20 min/level
- 1 hour/level → 2 hours/level
- Does NOT affect: Instantaneous, Concentration, Permanent, Discharged

Implementation (existing in `SpellCaster.ApplyMetamagicToSpellData()`):

```
if (metamagic.Has(MetamagicFeatId.ExtendSpell)) {
    if (spell.DurationValue > 0)
        spell.DurationValue *= 2;
    else if (spell.BuffDurationRounds > 0)
        spell.BuffDurationRounds *= 2;
}
```

5.4 Heighten Spell

Property	Value
PHB Page	p.88
Level Adjustment	Variable (target level – base level)
Prerequisite	None
Applicable To	Any spell
Effect	Increase the spell's effective level for ALL level-dependent effects

Detailed Mechanics:

- **Unlike all other metamagic:** Heighten Spell actually changes the spell's effective level
- Save DC recalculated: 10 + heightened level + ability modifier
- Affects: counterspelling (higher-level counterspell needed), globe of invulnerability interactions, save DCs

- Example: Hold Person (2nd level) heightened to 5th → Will DC increases by 3
- Level adjustment = target – base (variable cost)
- Must specify target level (3rd through 9th)

Implementation (existing in `SpellCaster.Cast()`):

```
bool isHeightened = metamagic != null && metamagic.Has(MetamagicFeatId.HeightenSpell);
int effectiveSpellLevel = spell.SpellLevel;
if (isHeightened && metamagic.HeightenToLevel > spell.SpellLevel)
    effectiveSpellLevel = metamagic.HeightenToLevel;
// DC = 10 + effectiveSpellLevel + abilityModifier
```

5.5 Maximize Spell

Property	Value
PHB Page	p.88
Level Adjustment	+3
Prerequisite	None
Applicable To	Spells with variable numeric effects (damage, healing)
Effect	All variable numeric effects use maximum possible values

Detailed Mechanics:

- Fireball (10d6) maximized → 60 damage (every d6 = 6)
- Cure Critical Wounds (4d8+15) maximized → 47 (32+15)
- Does NOT affect: saving throws, range, duration, number of targets
- Variable-and-bonus: maximize the dice, keep flat bonuses as-is

Implementation (existing in `SpellCaster.Cast()` — uses max dice values):

```
if (isMaximized) {
    // Use maximum values for all dice (e.g., d6 = 6, d8 = 8)
    totalDamage = maxDiceValue * numberOfDice + flatBonus;
}
```

5.6 Quicken Spell

Property	Value
PHB Page	p.88
Level Adjustment	+4
Prerequisite	None
Applicable To	Any spell
Effect	Casting time becomes a free action

Detailed Mechanics:

- Can cast a quickened spell AND another spell in the same round
- **Limit:** Only one quickened spell per round (already tracked by `HasQuickenedThisRound`)
- Casting a quickened spell does NOT provoke attacks of opportunity
- Spells with casting time > 1 standard action CANNOT be quickened
- This is the only metamagic that overrides the spontaneous caster casting time penalty

Implementation (existing in `SpellCaster.ApplyMetamagicToSpellData()`):

```
if (metamagic.Has(MetamagicFeatId.QuickenSpell))
    spell.ActionType = SpellActionType.Swift; // Treated as free action
```

5.7 Silent Spell

Property	Value
PHB Page	p.100
Level Adjustment	+1
Prerequisite	None
Applicable To	Any spell with verbal components
Effect	Removes verbal component requirement

Detailed Mechanics:

- Spell can be cast in a silence zone
- Spell can be cast while unable to speak
- A spell without verbal components gains no benefit (but can still be applied)
- Useful situationally: underwater, silence spell, gagged

Implementation (existing):

```
if (metamagic.Has(MetamagicFeatId.SilentSpell))
    spell.HasVerbalComponent = false;
```

5.8 Still Spell

Property	Value
PHB Page	p.100
Level Adjustment	+1
Prerequisite	None
Applicable To	Any spell with somatic components
Effect	Removes somatic component requirement

Detailed Mechanics:

- Spell can be cast while grappled (no somatic = no check needed)
- Spell can be cast while bound or restrained
- **Arcane Spell Failure:** Still spell bypasses armor's arcane spell failure chance (no gestures = no failure)
- Combined with Silent: completely invisible casting

Implementation (existing):

```
if (metamagic.Has(MetamagicFeatId.StillSpell))
    spell.HasSomaticComponent = false;
```

5.9 Widen Spell

Property	Value
PHB Page	p.102
Level Adjustment	+3
Prerequisite	None
Applicable To	Spells with burst, emanation, line, or spread area shapes
Effect	Double all numeric measurements of the spell's area

Detailed Mechanics:

- Fireball (20 ft radius) → 40 ft radius
- Cone of Cold (60 ft cone) → 120 ft cone
- Wall of Fire (20 ft + 5 ft/level) → 40 ft + 10 ft/level
- Does NOT affect number of targets for spells that target specific creatures
- Only works on spells with area measurements

Implementation (existing):

```
if (metamagic.Has(MetamagicFeatId.WidenSpell) && spell.AreaRadius > 0)
    spell.AreaRadius *= 2;
```

6. Technical Implementation

6.1 MetamagicData Enhancements

The existing `MetamagicData` class needs a few additions for rod support and validation:

```
// — ADDITIONS TO MetamagicData.cs —

/// <summary>
/// True if this metamagic is applied via a Metamagic Rod (bypasses slot increase).
/// </summary>
public bool IsFromRod;

/// <summary>
/// Reference to the rod item providing this metamagic (for charge/use tracking).
/// Null if metamagic is from a feat (not a rod).
/// </summary>
public ItemData RodSource;

/// <summary>
/// Set of metamagic feats applied via rod (these don't count toward slot increase).
/// </summary>
public HashSet<MetamagicFeatId> RodAppliedMetamagic = new HashSet<MetamagicFeatId>();

/// <summary>
/// Calculate the total spell level adjustment counting ONLY feat-applied metamagic.
/// Rod-applied metamagic does NOT increase slot level.
/// </summary>
public int GetFeatLevelAdjustment(int baseSpellLevel)
{
    int adj = 0;
    foreach (var mm in AppliedMetamagic)
    {
        if (!RodAppliedMetamagic.Contains(mm))
            adj += GetLevelAdjustment(mm, baseSpellLevel);
    }
    return adj;
}

/// <summary>
/// Get the effective slot level consumed (base + feat adjustments only).
/// Rod-applied metamagic is free.
/// </summary>
public int GetEffectiveSlotLevel(int baseSpellLevel)
{
    return baseSpellLevel + GetFeatLevelAdjustment(baseSpellLevel);
}

/// <summary>
/// Validate that the effective slot level does not exceed 9.
/// </summary>
public bool IsValid(int baseSpellLevel)
{
    return GetEffectiveSlotLevel(baseSpellLevel) <= 9;
}
```

6.2 SpellcastingComponent Metamagic Methods

New methods needed on `SpellcastingComponent` :

```
// — ADDITIONS TO SpellcastingComponent.cs —

/// <summary>
/// Get all metamagic feats this character has.
/// Reads from CharacterStats.Feats list, filters by IsMetamagic.
/// </summary>
public List<MetamagicFeatId> GetAvailableMetamagicFeats()
{
    var result = new List<MetamagicFeatId>();
    if (Stats?.Feats == null) return result;

    foreach (string featName in Stats.Feats)
    {
        MetamagicFeatId id = MetamagicData.GetIdFromFeatName(featName);
        if (id != MetamagicFeatId.None)
            result.Add(id);
    }
    return result;
}

/// <summary>
/// Check if character has a specific metamagic feat.
/// </summary>
public bool HasMetamagicFeat(MetamagicFeatId feat)
{
    string featName = MetamagicData.GetFeatName(feat);
    return Stats != null && Stats.Feats != null && Stats.Feats.Contains(featName);
}

/// <summary>
/// Check if a metamagic combination is valid for a spell.
/// Validates: feat owned, spell applicable, effective level ≤ 9, slot available.
/// </summary>
public bool CanApplyMetamagic(SpellData spell, MetamagicData metamagic)
{
    // Check each applied metamagic feat is owned
    foreach (var mm in metamagic.AppliedMetamagic)
    {
        if (!metamagic.RodAppliedMetamagic.Contains(mm) && !HasMetamagicFeat(mm))
            return false;
        if (!MetamagicData.IsApplicable(mm, spell))
            return false;
    }

    // Validate effective slot level ≤ 9
    int effectiveSlot = metamagic.GetEffectiveSlotLevel(spell.SpellLevel);
    if (effectiveSlot > 9)
        return false;

    // Check that a slot of the required level is available
    return HasAvailableSlot(effectiveSlot);
}

/// <summary>
/// Check if character has an unused spell slot at the given level.
/// </summary>
public bool HasAvailableSlot(int level)
{
    return SpellSlots.Any(s => s.Level == level && s.CanCast);
}

/// <summary>
```



```

/// For prepared casters: check if the caster is a spontaneous caster (Sorcerer,
Bard).
/// </summary>
public bool IsSpontaneousCaster()
{
    return Stats.HasClass("Sorcerer") || Stats.HasClass("Bard");
}

/// <summary>
/// Get the effective casting time after metamagic for spontaneous casters.
/// Prepared casters: no change. Spontaneous: standard → full-round.
/// Quicken always → free action regardless.
/// </summary>
public SpellActionType GetMetamagicCastingTime(SpellData spell, MetamagicData metamagic)
{
    // Quicken overrides everything
    if (metamagic.Has(MetamagicFeatId.QuickenSpell))
        return SpellActionType.Free;

    // Spontaneous casters: metamagic increases casting time
    if (IsSpontaneousCaster() && metamagic.HasAnyMetamagic)
    {
        if (spell.ActionType == SpellActionType.Standard)
            return SpellActionType.FullRound;
        // Full-round spells with metamagic = next-round casting (not supported in
        // current action system)
        // For now, treat as full-round (TODO: multi-round casting)
    }

    return spell.ActionType;
}

```

6.3 Prepared Caster Metamagic Preparation

For Wizards/Clerics, metamagic is applied at preparation time:

```
// — ADDITIONS TO SpellcastingComponent.cs —

/// <summary>
/// Prepare a metamagic version of a spell in a higher-level slot.
/// For prepared casters (Wizard, Cleric) only.
/// </summary>
public bool PrepareMetamagicSpell(SpellData baseSpell, MetamagicData metamagic, int targetSlotLevel)
{
    if (IsSpontaneousCaster())
    {
        Debug.LogWarning("[Metamagic] Spontaneous casters don't prepare metamagic versions.");
        return false;
    }

    int requiredLevel = metamagic.GetEffectiveSlotLevel(baseSpell.SpellLevel);
    if (requiredLevel != targetSlotLevel)
    {
        Debug.LogWarning($"[Metamagic] Slot level mismatch: need {requiredLevel}, got {targetSlotLevel}");
        return false;
    }

    // Find an empty slot at the target level
    SpellSlot targetSlot = SpellSlots.FirstOrDefault(
        s => s.Level == targetSlotLevel && s.IsEmpty && !s.LockedByImbue);

    if (targetSlot == null)
    {
        Debug.LogWarning($"[Metamagic] No empty slot at level {targetSlotLevel}");
        return false;
    }

    // Clone the spell and apply metamagic to the clone
    SpellData metamagicSpell = baseSpell.Clone();
    SpellCaster.ApplyMetamagicToSpellData(metamagicSpell, metamagic);

    // Store metamagic info on the spell clone for later reference
    metamagicSpell.AppliedMetamagic = metamagic; // New field on SpellData

    targetSlot.Prepare(metamagicSpell);
    return true;
}
```

6.4 SpellData Enhancement

Minor addition to track metamagic on prepared spell copies:

```
// — ADDITION TO SpellData.cs —

/// <summary>
/// Metamagic applied to this spell instance (null for unmodified spells).
/// Set when a prepared caster prepares a metamagic version.
/// Used to pass metamagic info through to SpellCaster.Cast().
/// </summary>
[System.NonSerialized]
public MetamagicData AppliedMetamagic;

/// <summary>
/// Create a deep clone of this SpellData for metamagic modification.
/// </summary>
public SpellData Clone()
{
    // MemberwiseClone handles all value types
    SpellData clone = (SpellData)this.MemberwiseClone();
    // Deep copy reference types if needed
    clone.AppliedMetamagic = null; // Clear on clone; will be set by caller
    return clone;
}
```

6.5 Empower + Maximize Stacking (PHB p.87)

When both Empower and Maximize are applied to the same spell:

“An empowered, maximized spell gains the separate benefits of each feat: the maximum result plus one-half the normally rolled result.”

```
// — IN SpellCaster.Cast() – DAMAGE RESOLUTION —

int baseDamage;
if (isMaximized && isEmpowered)
{
    // PHB: Maximize first, then add 50% of a normal roll
    int maximizedDamage = maxDiceValue * numberOfDice + flatBonus;
    int normalRoll = RollDice(numberOfDice, dieSize);
    int empowerBonus = Mathf.FloorToInt(normalRoll * 0.5f);
    baseDamage = maximizedDamage + empowerBonus;
    // Example: Maximized+Empowered Fireball (10d6):
    //   Maximized = 60, Normal roll = 35, Empower bonus = 17
    //   Total = 60 + 17 = 77
}
else if (isMaximized)
{
    baseDamage = maxDiceValue * numberOfDice + flatBonus;
}
else if (isEmpowered)
{
    int normalRoll = RollDice(numberOfDice, dieSize) + flatBonus;
    baseDamage = normalRoll + Mathf.FloorToInt(normalRoll * 0.5f);
}
else
{
    baseDamage = RollDice(numberOfDice, dieSize) + flatBonus;
}
```

7. Rod Integration Design

7.1 Overview

Metamagic Rods are wondrous items that apply metamagic effects to spells **without increasing the spell slot level**. They come in three tiers based on the maximum spell level they can affect:

Rod Tier	Max Spell Level	Approximate Price Range
Lesser	Up to 3rd-level spells	3,000 – 35,000 gp
Normal	Up to 6th-level spells	11,000 – 75,500 gp
Greater	Up to 9th-level spells	24,500 – 170,000 gp

All rods: **3 uses per day**, standard usage.

7.2 Rod Application Flow

```

Player selects spell
└─ Player toggles metamagic (from feat AND/OR rod)
    └─ System calculates effective slot:
        Slot = Base + FeatAdjustments (Rod adjustments = FREE)
    └─ System validates:
        ✓ Spell level ≤ rod's max level
        ✓ Rod has uses remaining today
        ✓ Effective slot ≤ 9
        ✓ Slot available
    └─ On cast:
        - Consume appropriate slot
        - Decrement rod uses
        - Apply ALL metamagic effects (feat + rod) to spell clone
        - Resolve normally
  
```

7.3 MetamagicData Rod Fields

```

// Already designed in Section 6.1:
public bool IsFromRod;
public ItemData RodSource;
public HashSet<MetamagicFeatId> RodAppliedMetamagic;
  
```

7.4 Rod ItemData Fields (Future)

```

// To be added to ItemData.cs when Rod system is implemented:
public bool IsMetamagicRod;
public MetamagicFeatId RodMetamagicType; // Which metamagic this rod provides
public string RodTier; // "Lesser", "Normal", "Greater"
public int RodMaxSpellLevel; // 3, 6, or 9
public int RodUsesPerDay; // Always 3 for metamagic rods
public int RodUsesRemainingToday; // Decrement on use
  
```

7.5 Rod Price Table (DMG pp. 236-237)

Metamagic	Lesser (≤ 3 rd)	Normal (≤ 6 th)	Greater (≤ 9 th)
Enlarge	3,000 gp	11,000 gp	24,500 gp
Extend	3,000 gp	11,000 gp	24,500 gp
Silent	3,000 gp	11,000 gp	24,500 gp
Still	3,000 gp	11,000 gp	24,500 gp
Empower	9,000 gp	32,500 gp	73,000 gp
Maximize	14,000 gp	54,000 gp	121,500 gp
Widen	14,000 gp	54,000 gp	121,500 gp
Quickened	35,000 gp	75,500 gp	170,000 gp

Note: There is **no** Rod of Heighten Spell (Heighten is variable-level, incompatible with rod tiers).

8. Stacking Rules

8.1 Multiple Metamagic on One Spell

Multiple different metamagic feats **can** be applied to the same spell. Their level adjustments are **cumulative**:

```

Empowered, Maximized Fireball:
  Base: 3rd level
  + Empower: +2
  + Maximize: +3
  = 8th-level slot required

Extended, Silent, Still Mage Armor:
  Base: 1st level
  + Extend: +1
  + Silent: +1
  + Still: +1
  = 4th-level slot required

```

8.2 Same Metamagic Twice

The **same** metamagic feat **cannot** be applied twice to the same spell. Each metamagic is either on or off.

Exception: There is no explicit rule preventing it, but the `HashSet<MetamagicFeatId>` implementation naturally prevents duplicates.

8.3 Feat + Rod Stacking

A caster **can** apply metamagic from both a feat and a rod to the same spell:

Example: Wizard with Empower Spell feat + Lesser Rod of Extend
Casting: Extended, Empowered Magic Missile (1st level)

Extend: from rod (FREE – rod handles ≤ 3 rd level spells ✓)

Empower: from feat (+2 level adjustment)

Slot consumed: $1 + 2 = 3$ rd-level slot

(Without rod: $1 + 1 + 2 = 4$ th-level slot)

8.4 Empower + Maximize Interaction

As detailed in Section 6.5:

- Maximize all dice to maximum values
- Then add 50% of a separately rolled normal result
- Level cost: +2 (Empower) + +3 (Maximize) = +5 total

8.5 Stacking Examples

Spell	Metamagic Applied	Slot Level	Notes
Magic Missile (1st)	Empower (+2)	3rd	1d4+1 → ×1.5 damage per missile
Magic Missile (1st)	Quicken (+4)	5th	Free action cast
Fireball (3rd)	Maximize (+3)	6th	10d6 → 60 damage
Fireball (3rd)	Empower (+2) + Maximize (+3)	8th	60 + 50% of normal roll
Fireball (3rd)	Quicken (+4) + Empower (+2)	9th	Free action + 50% damage
Fireball (3rd)	Quicken (+4) + Maximize (+3)	INVALID	3+4+3 = 10 > 9
Hold Person (2nd)	Heighten to 5th (+3)	5th	DC increases by 3
Cure Light (1st)	Extend (+1)	2nd	2× duration (if applicable)
Mage Armor (1st)	Extend (+1) + Still (+1)	3rd	2hr/level, no somatic
Fly (3rd)	Extend (+1) + Silent (+1) + Still (+1)	6th	2× duration, no components
Fireball (3rd)	Rod of Maximize (Lesser ≤3rd ✓)	3rd	Max damage, no slot increase
Disintegrate (6th)	Rod of Maximize (Lesser ≤3rd ✗)	INVALID	6th > 3 = rod can't apply

9. Spontaneous vs Prepared Casting

9.1 Prepared Casters (Wizard, Cleric, Druid, Paladin, Ranger)

When metamagic is applied: During daily spell preparation (morning ritual).

How it works:

1. Wizard decides to prepare “Empowered Fireball”
2. This fills a **5th-level slot** (3 + 2) with the metamagic version
3. At cast time, no additional decisions — just cast from the slot
4. The slot’s spell is already a modified SpellData clone
5. Casting time is **unchanged** (standard action for standard action spells)

Implementation approach: `SpellSlot` stores a cloned `SpellData` with metamagic already applied. The `MetamagicData` is stored on the clone for `SpellCaster.Cast()` to reference during resolution.

```
Morning Preparation:
  5th-level slot ← Empowered Fireball (SpellData clone with metamagic)

Combat:
  Cast from 5th-level slot → SpellCaster.Cast(clone, clone.AppliedMetamagic)
```

9.2 Spontaneous Casters (Sorcerer, Bard)

When metamagic is applied: At cast time (real-time decision).

How it works:

1. Sorcerer decides during combat to empower their Fireball
2. System checks: has Empower Spell feat? Has a 5th-level slot available?
3. Casting time increases: standard → **full-round action**
4. 5th-level spell slot is consumed
5. A fresh `SpellData` clone is created and metamagic applied on the spot

Critical difference: Spontaneous casters don't prepare specific spells in slots. They have a pool of slots per level and choose freely. So metamagic decisions happen at cast time, not preparation time.

Casting time penalty:

- Standard action spell + metamagic → **Full-round action**
- Exception: Quicken Spell → Free action (overrides the penalty)
- Full-round action spell + metamagic → **1 full round** (start next turn)

```
Combat (Sorcerer):
  1. Select: Fireball
  2. Toggle: Empower Spell (+2 levels)
  3. UI shows: "Uses 5th-level slot | Full-round action"
  4. Confirm → consumes 5th-level slot, full-round action
  5. SpellCaster.Cast(freshClone, newMetamagicData)
```


9.3 Implementation Differences Summary

Aspect	Prepared	Spontaneous
When chosen	Preparation time	Cast time
Stored where	SpellSlot.PreparedSpell (clone)	Created at cast time
Slot consumed	The higher-level slot it was prepared in	Any unused slot of required level
Casting time	Unchanged	Standard → Full-round
UI	Preparation screen metamagic toggle	Cast-time metamagic toggle
Cancel	Must re-prepare next day	Can decide each cast
Flexibility	Low (locked at prep time)	High (decide per cast)

9.4 Multiclass Caster Considerations

A character with both Wizard and Sorcerer levels:

- Wizard spells: prepared metamagic (no casting time change)
- Sorcerer spells: spontaneous metamagic (full-round action)
- Metamagic feats are shared across classes
- Spell slots are per-class (already handled by `CasterClassName` on `SpellSlot`)

10. Implementation Phases

Phase 1: Core Metamagic Query System (Est. 1-2 hours)

Priority: HIGH — Foundation for all other phases

Goal: Character can query available metamagic feats, validate combinations.

Files Modified:

- `SpellcastingComponent.cs` — Add `GetAvailableMetamagicFeats()` , `HasMetamagicFeat()` , `CanApply-Metamagic()`
- `FeatManager.cs` — Add `GetMetamagicFeats(CharacterStats)` helper

Deliverables:

- [x] Query character's metamagic feats from `CharacterStats.Feats`
- [x] Validate metamagic applicability per spell
- [x] Calculate effective slot level
- [x] Enforce level 9 cap
- [x] Unit test: Wizard with Empower gets correct available list
- [x] Unit test: Level 9 cap prevents invalid combinations

Phase 2: Prepared Caster Metamagic Preparation (Est. 2-3 hours)

Priority: HIGH — Wizard is the primary prepared caster

Goal: Wizards can prepare metamagic spell versions in higher-level slots.

Files Modified:

- `SpellcastingComponent.cs` — Add `PrepareMetamagicSpell()`
- `SpellData.cs` — Add `AppliedMetamagic` field and `Clone()` method
- `SpellPreparationUI.cs` — Add metamagic toggle during preparation

Deliverables:

- [] `SpellData.Clone()` for non-destructive metamagic application
- [] `PrepareMetamagicSpell()` puts modified clone in higher slot
- [] `SpellSlot` displays metamagic info in `ToString()`
- [] Preparation UI shows available metamagic per spell
- [] Preparation UI shows required slot level before confirming
- [] Unit test: Empowered Fireball consumes 5th-level slot
- [] Unit test: Cannot prepare metamagic if no slot at required level

Phase 3: Spontaneous Caster Metamagic (Est. 2-3 hours)

Priority: HIGH — Sorcerer is the primary spontaneous caster

Goal: Sorcerers can apply metamagic at cast time with casting time penalty.

Files Modified:

- `SpellcastingComponent.cs` — Add `IsSpontaneousCaster()`, `GetMetamagicCastingTime()`
- `CombatActions.cs` or `GameManager` casting flow — Enforce casting time changes
- `SpellcastingComponent.cs` — Metamagic-aware slot consumption for spontaneous casters

Deliverables:

- [] Spontaneous metamagic = full-round action (standard spells)
- [] Quicken overrides to free action for both caster types
- [] Consume correct slot level for spontaneous metamagic
- [] `HasQuickenedThisRound` enforcement (already exists)
- [] Unit test: Sorcerer metamagic uses full-round action
- [] Unit test: Quickened spell is free action for Sorcerer

Phase 4: Metamagic Selection UI (Est. 3-4 hours)

Priority: MEDIUM — Player-facing UI

Goal: Players can toggle metamagic feats in spell selection UI.

Files Modified:

- New: `MetamagicSelectionPanel.cs` — UI panel for metamagic toggles
- `SpellPreparationUI.cs` — Integrate metamagic into preparation flow
- `CombatUI.cs` or spell casting UI — Integrate metamagic into cast-time flow

Deliverables:

- [] Toggle button per available metamagic feat
- [] Greyed-out buttons for inapplicable metamagic (with tooltip why)
- [] Real-time display of effective slot level
- [] Warning when combination exceeds level 9

- [] Heighten Spell: dropdown to select target level (3-9)
- [] Casting time display for spontaneous casters
- [] Rod toggle (placeholder for Phase 7)

Phase 5: Empower/Maximize Resolution Polish (Est. 1-2 hours)

Priority: MEDIUM — Correctness for combat resolution

Goal: Fix Empower + Maximize stacking order and verify all damage calculations.

Files Modified:

- `SpellCaster.cs` — Refine Empower/Maximize resolution code

Deliverables:

- [] Maximize + Empower stacking: max dice + 50% of normal roll
- [] Empower correctly handles healing spells
- [] Maximize correctly handles variable bonus amounts
- [] Combat log shows metamagic effects clearly
- [] Unit test: Maximized+Empowered Fireball = correct damage
- [] Unit test: Empowered Cure Critical Wounds = correct healing

Phase 6: AI Metamagic Strategy (Est. 2-3 hours)

Priority: LOW — NPC intelligence enhancement

Goal: AI NPCs can intelligently apply metamagic feats.

Files Modified:

- `AISpellcastingStrategist.cs` — Add metamagic scoring
- `AIService.cs` — Pass metamagic to spell casting flow

Deliverables:

- [] AI scores: “Is empowering this spell worth the slot?”
- [] AI priority rules:
 - Quicken: When already casting another spell this turn
 - Empower/Maximize: For high-value damage spells against tough targets
 - Extend: For long-duration buffs before combat
 - Heighten: For save-or-die spells against high-save targets
 - Silent/Still: Only in specific conditions (silence zone, grappled)
- [] AI respects slot economy (don’t waste 9th-level slots on empowered cantrips)

Phase 7: Rod Integration (Est. 2-3 hours)

Priority: HIGH — Part of Rod system implementation

Goal: Metamagic rods apply metamagic without slot increase.

Files Modified:

- `MetamagicData.cs` — Add rod fields (designed in Section 6.1)
- `ItemData.cs` — Add rod-specific fields
- `MetamagicSelectionPanel.cs` — Add rod toggle alongside feat toggles
- `SpellcastingComponent.cs` — Rod-aware slot calculation

Deliverables:

- [] `IsFromRod` and `RodAppliedMetamagic` on `MetamagicData`
- [] Rod tier validation (spell level ≤ rod max)

- [] Rod daily use tracking and reset
- [] UI shows rod option alongside feat option
- [] Feat + rod stacking works correctly
- [] 3 uses/day enforcement
- [] Unit test: Rod of Extend on 3rd-level spell = no slot increase
- [] Unit test: Lesser rod rejects 4th-level spells

Phase 8: Edge Cases & Polish (Est. 1-2 hours)

Priority: LOW — Completeness

Goal: Handle all edge cases and polish the system.

Deliverables:

- [] Cantrip metamagic: level 0 + metamagic uses a slot (level 0 + adj)
- [] Metamagic feats display in character sheet feat summary
- [] Preparation screen shows metamagic-prepared spells distinctly
- [] Spell resistance check uses heightened level
- [] Counterspelling interactions with heightened spells
- [] Dispel check uses original spell level (not heightened)
- [] Globe of Invulnerability uses heightened level

Total Estimated Effort: 14-22 hours

Phase 1: Core Query System		1-2 hrs	[FOUNDATION]
Phase 2: Prepared Caster Prep		2-3 hrs	[HIGH]
Phase 3: Spontaneous Caster		2-3 hrs	[HIGH]
Phase 4: Selection UI		3-4 hrs	[MEDIUM]
Phase 5: Empower/Maximize Polish		1-2 hrs	[MEDIUM]
Phase 6: AI Strategy		2-3 hrs	[LOW]
Phase 7: Rod Integration		2-3 hrs	[HIGH - Rod System]
Phase 8: Edge Cases & Polish		1-2 hrs	[LOW]

11. Testing Requirements

11.1 Unit Tests — Metamagic Validation

Test ID	Description	Expected Result
MM-001	Wizard with Empower Spell queries available metamagic	Returns <code>[EmpowerSpell]</code>
MM-002	Character without metamagic feats queries	Returns empty list
MM-003	Empower applicability on Fireball (damage)	Returns <code>true</code>
MM-004	Empower applicability on Mage Armor (buff, no damage)	Returns <code>false</code>
MM-005	Enlarge applicability on Shield (self-only)	Returns <code>false</code>
MM-006	Extend applicability on Magic Missile (instantaneous)	Returns <code>false</code>
MM-007	Widen applicability on Fireball (area)	Returns <code>true</code>
MM-008	Widen applicability on Charm Person (single target)	Returns <code>false</code>
MM-009	Quicken on any spell	Returns <code>true</code>
MM-010	Silent on any spell	Returns <code>true</code>
MM-011	Still on any spell	Returns <code>true</code>
MM-012	Heighten on any spell	Returns <code>true</code>

11.2 Unit Tests — Level Calculation

Test ID	Description	Expected Result
MM-020	Fireball (3) + Empower (+2)	Effective level = 5
MM-021	Magic Missile (1) + Quicken (+4)	Effective level = 5
MM-022	Fireball (3) + Empower (+2) + Maximize (+3)	Effective level = 8
MM-023	Fireball (3) + Quicken (+4) + Maximize (+3)	Effective level = 10 → INVALID
MM-024	Hold Person (2) + Heighten to 5	Effective level = 5
MM-025	Hold Person (2) + Heighten to 5 + Silent (+1)	Effective level = 6
MM-026	Cantrip (0) + Empower (+2)	Effective level = 2
MM-027	Rod: Fireball (3) + Rod of Empower	Effective slot = 3 (rod free)
MM-028	Rod + Feat: Fireball + Rod of Extend + Feat Empower	Effective slot = 5 (3+0+2)
MM-029	Lesser Rod + 4th-level spell	INVALID (exceeds rod max)

11.3 Integration Tests — Spell Resolution

Test ID	Description	Expected Result
MM-040	Empowered Fireball (10d6) damage	Roll × 1.5 (rounded down)
MM-041	Maximized Fireball (10d6) damage	Always 60
MM-042	Maximized + Empowered Fireball	60 + floor(normal_roll × 0.5)
MM-043	Extended Mage Armor duration	2 hours/level instead of 1
MM-044	Enlarged Fireball range	Double normal range
MM-045	Widened Fireball area	Double radius
MM-046	Quickened Fireball action type	Free action
MM-047	Silent Fireball components	HasVerbalComponent = false
MM-048	Still Fireball components	HasSomaticComponent = false
MM-049	Heightened Hold Person DC	DC = 10 + heightened_level + ability_mod
MM-050	Sorcerer metamagic casting time	Standard → Full-round
MM-051	Sorcerer quickened casting time	Standard → Free (override)
MM-052	Wizard metamagic casting time	Standard → Standard (no change)
MM-053	Combat log shows metamagic	Log includes “Metamagic: Empower Spell”

11.4 Edge Case Tests

Test ID	Description	Expected Result
MM-060	Same metamagic twice	Cannot add (HashSet prevents)
MM-061	Metamagic on spell without required feat	CanApplyMetamagic returns false
MM-062	Effective level > 9	IsValid returns false
MM-063	Cantrip + Silent (+1) slot	Uses 1st-level slot
MM-064	Heighten to same level	No effect (adjustment = 0)
MM-065	Heighten to level below base	No effect (clamped to base)
MM-066	QuickenedThisRound + another quicken	Second quicken blocked
MM-067	Metamagic on spell-like ability	Not applicable (SLAs ≠ spells)
MM-068	Rod charges depleted + try to use	Rod option unavailable

Appendix: Quick Reference Tables

A1. Complete Metamagic Feat Summary

Feat	Adj	PHB p.	Applicable To	Modification	Resolved At
Empower	+2	87	Damage/ Healing spells	×1.5 numeric effects	Resolution
Enlarge	+1	88	Range > per- sonal/touch	×2 range	Pre-cast clone
Extend	+1	88	Duration > instant	×2 duration	Pre-cast clone
Heighten	var	88	Any spell	Increase DC/ level	Resolution
Maximize	+3	88	Damage/ Healing spells	Max dice val- ues	Resolution
Quicken	+4	88	Any (not > 1 std action)	Free action casting	Pre-cast clone
Silent	+1	100	Any w/ verbal	Remove verbal comp	Pre-cast clone
Still	+1	100	Any w/ so- matic	Remove so- matic comp	Pre-cast clone
Widen	+3	102	Area spells	×2 area di- mensions	Pre-cast clone

A2. Maximum Base Spell Level by Metamagic

Metamagic	Max Base for Valid 9th Slot
Enlarge (+1)	8th
Extend (+1)	8th
Silent (+1)	8th
Still (+1)	8th
Empower (+2)	7th
Maximize (+3)	6th
Widen (+3)	6th
Quicken (+4)	5th

A3. Common Metamagic Combinations

Combination	Total Adj	Max Base Spell	Use Case
Empower + Maximize	+5	4th	Maximum damage output
Extend + Silent + Still	+3	6th	Stealth long-duration buffs
Quicken + Empower	+6	3rd	Burst damage as free action
Quicken + Maximize	+7	2nd	Maximum guaranteed damage, free
Enlarge + Widen	+4	5th	Maximum battlefield coverage
Heighten (to 9) + Quicken	varies	5th	Irresistible save-or-die
Silent + Still	+2	7th	Completely silent casting

A4. Metamagic Rod Coverage

Level:	0	1	2	3	4	5	6	7	8	9
Lesser:	✓	✓	✓	✓	x	x	x	x	x	x
Normal:	✓	✓	✓	✓	✓	✓	✓	x	x	x
Greater:	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

A5. Existing Code File Index

File	Path	Relevance
MetamagicData.cs	Assets/Scripts/Magic/Components/MetamagicData.cs	✓ Core metamagic data (COMPLETE)
SpellCaster.cs	Assets/Scripts/Magic/SpellCaster.cs	✓ Resolution logic (MOSTLY COMPLETE)
FeatDefinitions.cs	Assets/Scripts/Character/FeatDefinitions.cs	✓ All 9 feats defined (COMPLETE)
Feat.cs	Assets/Scripts/Character/Feat.cs	✓ FeatBenefit.IsMetamagic (COMPLETE)
SpellSlot.cs	Assets/Scripts/Magic/SpellSlot.cs	⚠ Needs metamagic awareness
SpellData.cs	Assets/Scripts/Magic/SpellData.cs	⚠ Needs Clone() and AppliedMetamagic
SpellcastingComponent.cs	Assets/Scripts/Magic/Components/SpellcastingComponent.cs	⚠ Needs metamagic query/validation
FeatManager.cs	Assets/Scripts/Character/FeatManager.cs	⚠ Needs metamagic helpers
ItemData.cs	Assets/Scripts/Inventory/ItemData.cs	🔮 Future: rod fields
AISpellcastingStrategist.cs	Assets/Scripts/Services/AISpellcastingStrategist.cs	🔮 Future: AI metamagic

Document created: May 25, 2026

For: D&D 3.5e Prototype — Metamagic Feats System

Next: Rod System Implementation (depends on Phases 1-3 of this plan)